

Curso de C++

Capítulo 1

Variables

Nivel: Desde principiante hasta nivel práctico

Enfoque: Programación y Desarrollo de Software en C++

Material técnico, educativo y profesional

Objetivo del capítulo

En este capítulo aprenderás qué son las variables en C++, por qué existen y cómo utilizarlas para representar información que cambia durante la ejecución de un programa. Partiremos de ejemplos muy simples y avanzaremos hasta escenarios cercanos al desarrollo de software real, como el monitoreo de máquinas, sensores, inventarios y sistemas de producción.

Al finalizar este capítulo serás capaz de

- Explicar con tus propias palabras qué es una variable y qué problema resuelve.
- Declarar, inicializar y modificar variables utilizando una sintaxis correcta en C++.
- Seleccionar tipos de datos básicos según la información que necesite un programa.
- Distinguir entre declaración, inicialización y asignación.
- Usar constantes cuando un valor no debe cambiar.
- Crear nombres claros y mantenibles siguiendo convenciones profesionales.
- Analizar errores frecuentes relacionados con tipos, valores y variables sin inicializar.
- Modelar información de situaciones reales mediante variables y combinar varias de ellas en un programa.

Introducción al tema

¿Qué son las variables?

Una variable es un nombre que utilizamos para representar un dato almacenado por el programa. Ese dato puede cambiar durante la ejecución. Una analogía útil es una caja etiquetada: la etiqueta permite identificarla y dentro se guarda información. En programación, el nombre identifica la variable y el tipo de dato describe qué clase de información puede representar.

¿Por qué existen?

Los programas necesitan recordar información: una temperatura, el número de piezas producidas, el nombre de un operador o el saldo de una cuenta. Sin variables, el programa tendría que trabajar únicamente con valores escritos directamente en el código, lo que impediría adaptar fácilmente el comportamiento a datos nuevos.

¿Qué problema resuelven?

Resuelven la necesidad de almacenar y reutilizar información. Una misma variable puede participar en cálculos, decisiones, salidas por pantalla y posteriormente en funciones, estructuras de datos u objetos.

¿Dónde se utilizan?

Prácticamente en todo software: videojuegos para guardar puntos y vidas; sistemas industriales para temperaturas y velocidades; aplicaciones empresariales para clientes y ventas; robots para posición y estado; y sistemas de inventario para cantidades y precios.

¿Qué ocurriría si no existieran?

El código dependería de valores fijos. Para cambiar una temperatura, una cantidad o un nombre habría que modificar múltiples partes del programa. Las variables hacen posible que un programa trabaje con información dinámica.

Relación con otros conceptos del curso

Las variables se conectan con operadores, entrada y salida, condiciones, ciclos, funciones, arreglos y programación orientada a objetos. Antes de decidir con `if`, repetir con `for` o encapsular datos en una clase, necesitas comprender qué información estás representando y cómo se almacena.

1. El concepto fundamental

¿Qué es una variable?

Es una entidad identificada por un nombre, asociada a un tipo de dato y utilizada para almacenar un valor durante la ejecución de un programa.

¿Por qué se utiliza?

Para evitar repetir valores literales y permitir que la información pueda cambiar, reutilizarse y procesarse.

¿Cuándo se utiliza?

Cuando el programa necesita representar información cuyo valor puede variar: contadores, mediciones, nombres, estados, resultados de cálculos o datos proporcionados por el usuario.

¿Cómo funciona?

Cuando declaras una variable, le indicas al compilador el tipo de información que vas a manejar y el nombre con el que la referenciarás. Después puedes asignarle un valor compatible. Conceptualmente, el programa mantiene un estado asociado a ese nombre durante el tiempo de vida de la variable.

Sintaxis o estructura

```
tipo nombre;  
tipo nombre = valor;  
nombre = nuevoValor;
```

tipo indica la clase de información; **nombre** identifica la variable; y **valor** es el dato que se almacena.

Nivel 1 — Ejemplo básico

```
#include <iostream>  
using namespace std;  
  
int main() {  
    int piezas = 25;  
  
    cout << piezas << endl;
```

```
return 0;
}
```

Explicación línea por línea

- **#include <iostream>**: permite utilizar herramientas de entrada y salida como cout.
- **using namespace std;**: permite escribir cout en lugar de std::cout en estos ejemplos.
- **int main()**: función principal desde la que comienza el programa.
- **int piezas = 25;**: declara una variable llamada piezas, de tipo entero, y la inicializa con 25.
- **cout << piezas << endl;**: muestra el valor almacenado actualmente.
- **return 0;**: indica que el programa terminó correctamente.

Resultado esperado: **25**. Observa que se imprime el valor contenido, no la palabra piezas.

2. Declaración, inicialización y asignación

¿Qué es cada proceso?

Declaración: crear una variable indicando su tipo y nombre. **Inicialización**: darle un valor al momento de crearla. **Asignación**: cambiar o establecer su valor después de que ya existe.

```
int temperatura; // Declaración
temperatura = 30; // Asignación

int velocidad = 1200; // Declaración + inicialización
velocidad = 1500; // Nueva asignación
```

Una variable puede recibir distintos valores compatibles con su tipo. Después de la última línea, velocidad contiene 1500, no 1200 y 1500 al mismo tiempo.

¿Por qué es importante distinguirlos?

Porque muchos errores aparecen cuando se intenta utilizar una variable antes de asignarle un valor válido o cuando se confunde el momento en que la variable se crea con el momento en que su contenido cambia.

3. Tipos de datos básicos

El tipo informa qué clase de valores representa una variable y qué operaciones son apropiadas. C++ posee muchos tipos; para comenzar, los más utilizados son los siguientes.

Tipo	Representa	Ejemplo	Uso frecuente
int	Enteros	42	Cantidad de piezas, edad, contador
double	Decimales	36.75	Temperatura, precio, medición
char	Un carácter	'A'	Código corto, categoría
bool	Verdadero/falso	true	Estado, validación, alarma
string	Texto	"Motor 01"	Nombres y descripciones

Nota: para utilizar string normalmente se incluye `#include <string>`. En muchos entornos también puede estar disponible indirectamente, pero incluir explícitamente lo que usas mejora la claridad.

int

Se utiliza para números enteros, es decir, valores sin parte decimal.

```
int piezasProducidas = 125;
int empleados = 8;
```

double

Representa números con parte decimal. Es apropiado para muchas mediciones y cálculos donde se requiere precisión decimal.

```
double temperatura = 36.5;
double precio = 149.99;
```

char

Almacena un solo carácter y se escribe entre comillas simples.

```
char categoria = 'A';
```

bool

Representa un estado lógico: true o false.

```
bool maquinaActiva = true;
bool alarmaEncendida = false;
```

string

Representa una secuencia de caracteres, normalmente utilizada para nombres, mensajes o descripciones.

```
#include <string>

string nombreMaquina = "Cortadora CNC";
```

4. Elegir el tipo correcto

¿Por qué importa?

El tipo debe representar adecuadamente el dominio del problema. Si la temperatura puede ser 24.5, int perdería la parte decimal. Si una información solo tiene dos estados, bool comunica mejor la intención que un número arbitrario.

Información	Tipo recomendado	Razón
Cantidad de productos	int	Normalmente es un conteo entero.
Temperatura de sensor	double	Puede incluir decimales.

Nombre de cliente	string	Es texto.
Máquina activa	bool	Solo existen dos estados principales.
Categoría de producto	char o string	Depende de si es un símbolo o una descripción.

5. Nombres de variables

¿Qué es un identificador?

El identificador es el nombre con el que el código hace referencia a una variable. Un buen nombre explica el propósito del dato.

- Puede contener letras, números y el carácter `_`.
- No puede comenzar con un número.
- No puede contener espacios.
- No debe utilizar una palabra reservada del lenguaje.
- C++ distingue mayúsculas y minúsculas: `temperatura` y `Temperatura` son nombres diferentes.

```
int piezasProducidas = 100; // Claro
double temperaturaActual = 28.4;

// Menos recomendable:
int x = 100;
double t = 28.4;
```

Los nombres cortos no siempre son incorrectos, pero en código que debe mantenerse durante meses o años, los nombres descriptivos reducen errores y tiempo de lectura.

6. Modificar el valor de una variable

¿Cómo funciona?

Una asignación reemplaza el valor anterior por uno nuevo. El nombre sigue identificando la misma variable dentro de su ámbito.

```
int piezas = 10;

piezas = 15;
piezas = piezas + 5;
```

Después de la primera modificación, `piezas` vale 15. En la última línea, primero se evalúa la expresión `piezas + 5` usando el valor actual y después el resultado se asigna nuevamente: el valor final es 20.

Diagrama conceptual

Estado inicial → **piezas = 10**

Asignación → **piezas = 15**

Cálculo + asignación → **15 + 5 → piezas = 20**

7. Constantes

¿Qué son?

Una constante representa un valor que no debe modificarse después de inicializarse. En C++ puede declararse con la palabra clave `const`.

```
const int MAXIMO_PIEZAS = 500;
const double TEMPERATURA_MAXIMA = 80.0;
```

Las constantes hacen explícito que ciertos valores forman parte de una regla o configuración que no debe cambiar accidentalmente. Son útiles para límites, capacidades máximas, conversiones y otros valores fijos.

```
const int MAXIMO_PIEZAS = 500;
// MAXIMO_PIEZAS = 600; // Error: no puede modificarse
```

Ejemplos progresivos

Nivel 2 — Ejemplo intermedio

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string nombre = "Ana";
    int piezas = 40;
    double temperatura = 31.8;
    bool maquinaActiva = true;

    cout << "Operador: " << nombre << endl;
    cout << "Piezas: " << piezas << endl;
    cout << "Temperatura: " << temperatura << endl;
    cout << "Activa: " << maquinaActiva << endl;

    return 0;
}
```

Este ejemplo integra distintos tipos para modelar una pequeña parte del estado de un sistema. El valor booleano puede mostrarse como 1 o 0 por defecto, dependiendo de la configuración de salida; más adelante aprenderás formas más expresivas de mostrar estados.

Errores comunes del ejemplo

- Escribir Ana sin comillas: el compilador no lo interpreta como string.
- Usar comillas dobles para char: "A" es texto, mientras que 'A' representa un carácter.
- Intentar guardar 31.8 en int esperando conservar el decimal.

Nivel 3 — Ejemplo práctico: monitoreo de una máquina

```
#include <iostream>
#include <string>
using namespace std;

int main() {
```

```

string codigoMaquina = "CNC-01";
double temperatura = 68.5;
int piezasProducidas = 245;
bool activa = true;

cout << "=== ESTADO DE MAQUINA ===" << endl;
cout << "Codigo: " << codigoMaquina << endl;
cout << "Temperatura: " << temperatura << " C" << endl;
cout << "Piezas producidas: " << piezasProducidas << endl;
cout << "Estado: " << activa << endl;

return 0;
}

```

Análisis

Cada variable representa una propiedad diferente. No todas deberían utilizar el mismo tipo: el código es texto, la temperatura puede tener decimales, la producción es un conteo y el estado es lógico. Esta elección comunica el significado del sistema.

Resultado esperado: un reporte con los valores actuales de la máquina. Si la temperatura cambia posteriormente, bastará con actualizar la variable correspondiente y el resto del código podrá utilizar el nuevo valor.

Nivel 4 — Ejemplo profesional: estado y cálculo de producción

```

#include <iostream>
#include <string>
using namespace std;

int main() {
    const int META_DIARIA = 1000;

    string lineaProduccion = "Linea A";
    int piezasInicio = 720;
    int piezasNuevas = 135;
    int piezasActuales = piezasInicio + piezasNuevas;
    double porcentajeMeta =
        (static_cast<double>(piezasActuales) / META_DIARIA) * 100.0;
    bool metaAlcanzada = piezasActuales >= META_DIARIA;

    cout << "Linea: " << lineaProduccion << endl;
    cout << "Produccion actual: " << piezasActuales << endl;
    cout << "Avance: " << porcentajeMeta << "%" << endl;
    cout << "Meta alcanzada: " << metaAlcanzada << endl;

    return 0;
}

```

Explicación progresiva

El ejemplo utiliza variables para representar datos de entrada, resultados derivados y un estado lógico.

META_DIARIA es constante porque representa una regla fija para esta ejecución. **piezasActuales** se calcula a partir de otras variables. **static_cast<double>** convierte temporalmente el valor para que el cálculo del porcentaje conserve decimales. **metaAlcanzada** almacena el resultado de una comparación.

En una aplicación mayor, estas variables podrían estar dentro de funciones, clases u objetos. El principio sigue siendo el mismo: elegir nombres y tipos que representen correctamente el estado del problema.

Comparaciones importantes

Concepto	Qué es	Cuándo usarlo	Diferencia principal
Variable	Dato que puede cambiar	Información dinámica	Su valor puede modificarse.
Constante	Dato que no debe cambiar	Reglas y límites fijos	No se reasigna después de inicializarse.
Declaración	Creación con tipo y nombre	Antes de usar la variable	No necesariamente asigna valor.
Inicialización	Primer valor al crear	Al declarar	Ocurre en el momento de creación.
Asignación	Cambio o establecimiento de valor	Durante el programa	Puede ocurrir varias veces.

Errores de confusión comunes

- **Variable vs. valor:** una variable es el elemento identificado por un nombre; el valor es la información que contiene en un momento determinado.
- **= en asignación:** en una expresión como `x = 10`, el valor 10 se almacena en x. No significa igualdad matemática.
- **int vs. double:** int representa enteros; double permite representar números con parte decimal.
- **char vs. string:** char almacena un carácter; string representa una secuencia de caracteres.
- **Declarar vs. inicializar:** declarar crea la variable; inicializar proporciona su primer valor.

Errores comunes al trabajar con Variables

1. Usar una variable sin inicializar

```
int piezas;  
cout << piezas; // Valor indeterminado: error lógico potencial
```

Qué ocurre: la variable se creó, pero no se le asignó un valor válido antes de utilizarla. **Consecuencia:** el programa puede mostrar o procesar un valor indeterminado. **Corrección:** inicializa cuando sea posible: `int piezas = 0;` **Prevención:** adopta la inicialización como práctica habitual.

2. Elegir un tipo incorrecto

```
int temperatura = 25.8; // La parte decimal no representa correctamente el dato esperado
```

Si el problema necesita decimales, utiliza un tipo adecuado como double. La prevención consiste en analizar qué representa cada dato antes de escribir el código.

3. Nombres poco descriptivos

Variables como a, b o x pueden dificultar la comprensión. En cálculos pequeños pueden ser aceptables, pero en proyectos reales nombres como `temperaturaActual` o `totalVentas` reducen ambigüedad.

4. Intentar modificar una constante

Si un valor debe cambiar, no debe modelarse como const. Si no debe cambiar, const protege la intención del diseño.

5. Confundir mayúsculas y minúsculas

```
int piezas = 10;
// cout << Piezas; // Error: Piezas no es piezas
```

C++ es sensible a mayúsculas y minúsculas. Mantén una convención consistente.

Buenas prácticas

- **Inicializa las variables:** reduce el riesgo de utilizar valores indeterminados.
- **Elige el tipo según el dominio:** el tipo debe representar correctamente la información.
- **Usa nombres descriptivos:** el código se lee muchas más veces de las que se escribe.
- **Prefiere const para valores que no deben cambiar:** comunica intención y evita modificaciones accidentales.
- **Mantén una convención:** por ejemplo, camelCase para variables como piezasProducidas.
- **Declara cerca del lugar de uso cuando sea razonable:** facilita comprender el ciclo de vida del dato.
- **Evita reutilizar una variable para significados distintos:** una variable llamada temperatura no debería después representar cantidad de piezas.
- **Comenta el porqué, no lo obvio:** un buen nombre suele eliminar la necesidad de comentarios redundantes.

Conectando conocimientos

Las variables se apoyan en conocimientos anteriores como sintaxis, salida con cout y tipos de datos. A partir de aquí se conectarán con operadores para realizar cálculos, con cin para recibir información, con if para tomar decisiones y con ciclos para actualizar datos repetidamente. Más adelante, arreglos permitirán almacenar colecciones y las clases agruparán variables y comportamientos para representar entidades completas.

Progresión: **valores literales** → **variables** → **operaciones** → **decisiones** → **repetición** → **funciones** → **colecciones** → **objetos**.

Ejercicios prácticos

Ejercicio 1 — Comprensión básica

Objetivo: declarar y mostrar variables.

Enunciado: crea un programa que almacene tu nombre, tu edad y la cantidad de horas que estudias por semana.

Requisitos: utiliza tipos apropiados y muestra cada dato con una etiqueta clara.

Pistas: pregunta primero si el dato es texto, entero o decimal; inicializa cada variable.

Ejercicio 2 — Aplicación

Objetivo: seleccionar tipos adecuados.

Contexto: un almacén necesita registrar el nombre de un producto, cantidad disponible, precio unitario y si está disponible para venta.

Requisitos: declara las variables con tipos apropiados y muestra un reporte.

Restricciones: no utilices arreglos ni estructuras todavía.

Pistas: identifica qué datos son texto, conteos, decimales y estados.

Ejercicio 3 — Integración

Objetivo: combinar variables con operadores básicos.

Problema: una línea inicia con cierta cantidad de piezas y produce una cantidad adicional. Calcula y muestra la producción final.

Requisitos: utiliza variables separadas para los valores de entrada y para el resultado.

Pistas: evita sobrescribir un dato si todavía necesitas conservar su valor original.

Ejercicio 4 — Caso práctico

Contexto: crea el estado básico de un robot industrial.

Requisitos: representa código, posición o zona, velocidad, cantidad de ciclos y estado activo/inactivo. Muestra un reporte organizado.

Pistas: elige cada tipo por el significado del dato, no por comodidad.

Ejercicio 5 — Reto

Problema: diseña la representación inicial de un sistema de producción que controle una meta diaria, producción inicial, piezas nuevas, producción actual, porcentaje de avance y estado de cumplimiento.

Reglas: usa una constante para la meta; conserva las variables intermedias; calcula el porcentaje con decimales.

Funcionalidades esperadas: reporte completo y valores claramente identificados.

Pistas progresivas: 1) modela los datos; 2) identifica cuáles cambian; 3) separa entradas y resultados; 4) revisa si una división entre enteros podría perder decimales.

Mini proyecto del capítulo

Nombre del proyecto

Monitor básico de producción

Contexto

Una pequeña planta necesita una primera versión de un programa de consola que represente el estado actual de una estación de trabajo.

Objetivo

Aplicar variables, tipos de datos, constantes, asignaciones y operaciones sencillas para generar un reporte de producción.

Requisitos funcionales

- Registrar un identificador de máquina.
- Representar la temperatura actual.
- Registrar piezas iniciales y piezas producidas durante el periodo.
- Calcular el total actual.
- Representar si la máquina está activa.
- Definir una meta de producción que no pueda modificarse.
- Mostrar un reporte claro.

Requisitos técnicos

- Utilizar al menos string, int, double y bool cuando tengan sentido.
- Utilizar al menos una constante con const.
- Usar nombres descriptivos.
- Inicializar las variables antes de utilizarlas.

Paso 1

Define qué información necesita conocer el sistema y crea las variables con el tipo más adecuado. Antes de programar, escribe una lista de datos y clasifícalos.

Paso 2

Inicializa los valores y muestra un primer reporte. Comprueba que cada etiqueta corresponde al dato correcto.

Paso 3

Agrega el cálculo de producción actual y el porcentaje respecto a la meta. Piensa cuidadosamente en qué variable debe almacenar cada resultado.

Paso final

Integra todo en un programa funcional y revisa nombres, tipos e inicialización. **Pista:** separa datos de entrada, reglas constantes y resultados calculados.

Comprueba tus conocimientos

1. ¿Qué es una variable y qué problema resuelve?
2. Explica la diferencia entre declaración, inicialización y asignación.

3. ¿Qué tipo utilizarías para almacenar una temperatura de 23.75? ¿Por qué?
4. ¿Qué diferencia existe entre char y string?
5. ¿Qué ocurre conceptualmente después de ejecutar: `int x = 5; x = x + 3;`?
6. ¿Por qué puede ser peligroso utilizar una variable local sin inicializar?
7. Analiza: `int piezas = 10; double total = piezas / 4;` ¿Qué resultado podría producir y por qué?
8. ¿Cuándo conviene utilizar const?
9. ¿Cuál nombre comunica mejor la intención: `x` o `temperaturaMotor`? ¿En qué contexto podría `x` ser aceptable?
10. ¿Por qué `maquinaActiva` y `MaquinaActiva` pueden representar identificadores diferentes?
11. Corrige conceptualmente: `char letra = "A";`
12. Diseña variables adecuadas para código de cliente, saldo, cantidad de pedidos y estado de cuenta.
13. ¿Qué ventaja tiene almacenar una meta fija en una constante?
14. En un sistema de producción, ¿por qué sería útil conservar `piezasIniciales` y `piezasNuevas` en lugar de usar una única variable para todo?
15. Explica cómo las variables preparan el camino para condiciones, ciclos, funciones y objetos.

Respuestas y explicaciones

1. Una variable representa un dato mediante un nombre y un tipo. Permite almacenar, reutilizar y, normalmente, modificar información durante la ejecución.
2. La declaración introduce tipo y nombre; la inicialización proporciona el primer valor al crearla; la asignación establece o reemplaza un valor posteriormente.
3. `double`, porque la medición contiene una parte decimal que `int` no representa de la misma manera.
4. `char` representa un carácter individual con comillas simples; `string` representa texto compuesto por una secuencia de caracteres.
5. `x` comienza en 5; después se calcula $5 + 3$ y el resultado 8 reemplaza el valor anterior.
6. Porque su contenido puede no ser un valor válido para el cálculo o comportamiento esperado. Inicializar reduce este riesgo.
7. La división `piezas / 4` se realiza entre enteros y puede producir un resultado entero antes de almacenarse en `double`. Si se necesitan decimales, debe intervenir una conversión adecuada.
8. Cuando un valor representa una regla, límite o dato que no debe modificarse después de inicializarse.
9. `temperaturaMotor` comunica claramente el propósito. `x` puede ser aceptable en contextos matemáticos muy pequeños donde su significado sea inmediato.
10. C++ distingue mayúsculas y minúsculas; por tanto, esos identificadores no son iguales.
11. Debe utilizarse `char letra = 'A';` porque `char` representa un solo carácter.

- 12.** Por ejemplo: string para código, double para saldo, int para pedidos y bool para un estado binario, siempre verificando que el dominio real del sistema coincida.
- 13.** Comunica que la meta no debe modificarse y permite reutilizar un nombre significativo en lugar de repetir un número literal.
- 14.** Porque cada variable conserva una parte del estado y facilita auditoría, cálculos y comprensión del origen del resultado.
- 15.** Las condiciones evalúan valores, los ciclos actualizan variables, las funciones reciben y producen datos, y los objetos agrupan variables que describen su estado.

Resumen del capítulo

Concepto	Para qué sirve	Cuándo utilizarlo	Error común
Variable	Representar datos	Cuando el programa necesita información	Usarla sin inicializar
Tipo de dato	Definir la clase de información	Al modelar cada dato	Elegirlo por costumbre
Inicialización	Dar un primer valor válido	Al crear una variable	Confundirla con asignación
Asignación	Actualizar contenido	Cuando cambia el estado	Perder valores necesarios
const	Proteger valores fijos	Reglas y límites	Intentar modificarla
Nombre descriptivo	Comunicar intención	Siempre que sea posible	Usar nombres ambiguos

Ideas clave que debes recordar

- Una variable representa información mediante un nombre y un tipo.
- El tipo debe elegirse según el significado y los valores esperados.
- Declarar, inicializar y asignar son acciones relacionadas, pero diferentes.
- Una asignación puede reemplazar el valor anterior.
- Inicializar variables mejora la seguridad y previsibilidad del programa.
- const comunica que un valor no debe modificarse.
- Los nombres descriptivos hacen el código más fácil de mantener.
- int, double, char, bool y string cubren muchos casos iniciales.
- Las variables son la base para cálculos, decisiones, ciclos, funciones y objetos.
- Modelar correctamente los datos es una habilidad fundamental en el desarrollo de software.

Mapa de aprendizaje

Conocimiento previo → sintaxis, salida y valores literales

Nuevo concepto → variables, tipos, inicialización y asignación

Aplicación básica → almacenar y mostrar datos

Aplicación práctica → representar máquinas, sensores y producción

Integración en proyectos → cálculos, estados y reglas constantes

Uso profesional → modelado claro, mantenible y escalable de información

El siguiente avance natural es utilizar operadores y entrada de datos para que las variables no solo almacenen valores definidos por el programador, sino que participen activamente en cálculos y reciban información dinámica. Las variables que aprendiste aquí serán los bloques básicos de prácticamente todos los programas que construyas en C++.